AI & Machine Learning — Task 4
Classification Models, Evaluation Metrics & Handling Imbalanced Data

1. INTRODUCTION

This report documents the complete Machine Learning classification workflow for Task 4 of the Maincrafts Technology AI & ML Internship. This task represents the most significant conceptual shift in the entire internship — moving from regression (predicting a number) to classification (predicting a category). While Tasks 1 through 3 focused entirely on predicting California house prices as a continuous value, Task 4 addresses one of the most common and impactful real-world AI problem types: binary classification.

Classification problems appear in virtually every industry — spam filtering, fraud detection, customer churn prediction, loan default assessment, and medical diagnosis. In all these cases, the model must decide between two or more categories rather than compute a continuous value. Task 4 builds a cancer diagnosis system using the Breast Cancer Dataset, trains three classification models, applies proper evaluation metrics beyond simple accuracy, handles class imbalance professionally, and selects the final model with complete scientific and clinical justification.

This project covers every step of the professional classification ML lifecycle:

- Loading and analyzing the Breast Cancer Dataset with class distribution examination
- Applying stratified train-test split to preserve class proportions in both sets
- Normalizing all 30 features using StandardScaler to ensure fair learning
- Training Logistic Regression as the interpretable baseline classification model
- Analyzing the Confusion Matrix to understand True and False Positives and Negatives
- Computing and interpreting Precision, Recall, F1-Score, and ROC-AUC
- Handling class imbalance using class_weight="balanced" technique
- Comparing Logistic Regression against Decision Tree Classifier on all five metrics
- Selecting the final model with clinical and statistical justification
- Visualizing results through five professional charts

2. PROBLEM STATEMENT

Question: Can a Machine Learning model correctly classify breast tumors as Malignant (cancerous) or Benign (safe) from 30 tumor measurement features — and can we evaluate that model using metrics that genuinely reflect medical reliability rather than misleading accuracy scores?

This is a supervised binary classification problem. Binary means exactly two possible output classes: 0 (Malignant) and 1 (Benign). The model learns from 455 labeled patient records during training and must generalize to classify 114 new unseen patients correctly.

Why does this problem matter?

In medical diagnosis specifically:

- A False Positive (predicting Benign when actually Malignant) means a cancer patient receives no treatment — potentially fatal
- A False Negative (predicting Malignant when actually Benign) causes unnecessary treatment and patient anxiety — costly and distressing but survivable
- This critical asymmetry in error cost proves that accuracy alone is completely insufficient — Recall must be the primary evaluation metric

Why does class imbalance matter here?

The dataset contains 212 Malignant (37.3%) and 357 Benign (62.7%) cases. A naive model that always predicts Benign would achieve 62.7% accuracy — a seemingly reasonable number — while detecting absolutely zero cancer cases. This is the fundamental danger of using accuracy as the sole metric on imbalanced classification problems.

What makes this challenging?

The model must correctly identify cancer cases despite having fewer training examples of them. It must balance the trade-off between Recall (catching every cancer) and Precision (minimizing false alarms) — a balance that carries direct clinical consequences for real patients.

3. SOLUTION APPROACH

We apply a seven-stage professional classification pipeline: load and analyze data, split with stratification, scale features, train and evaluate baseline model with proper metrics, generate ROC curve, handle imbalance, compare with Decision Tree, and select the final model with complete justification.

Step-by-step approach

Step	Action	Tool Used
1	Import all required libraries	pandas, numpy, sklearn, matplotlib, seaborn
2	Load dataset and analyze class distribution	sklearn.datasets, pandas
3	Stratified train-test split 80/20	train_test_split (stratify=y)
4	Feature scaling — all 30 features normalized	StandardScaler
5	Train baseline Logistic Regression	LogisticRegression (max_iter=1000)

Step	Action	Tool Used
6	Confusion Matrix analysis and visualization	confusion_matrix, seaborn heatmap
7	Precision, Recall, F1-Score evaluation	classification_report, sklearn.metrics
8	ROC Curve and AUC Score for all models	roc_curve, roc_auc_score
9	Class imbalance handling	class_weight="balanced"
10	Decision Tree Classifier comparison	DecisionTreeClassifier (max_depth=5)
11	Final model selection and justification	Complete written analysis

Each step is explained in full detail in the following sections.

4. DATASET OVERVIEW

The Breast Cancer Dataset is a classic, well-known medical dataset built directly into scikit-learn. It contains measurements computed from digitized images of fine needle aspirate (FNA) of breast masses — 30 features describing characteristics of the cell nuclei present in each image. The dataset was created at the University of Wisconsin and has been a benchmark classification dataset since 1995.

How to load the dataset in Python

Python:

```
from sklearn.datasets import load_breast_cancer
import pandas as pd
data = load_breast_cancer()
X = pd.DataFrame(data.data, columns=data.feature_names)
y = pd.Series(data.target, name='Diagnosis')
print(X.shape)
# Output: (569, 30)
print(data.target_names)  # ['malignant' 'benign']
print(y.value_counts())   # 1: 357,  0: 212
```

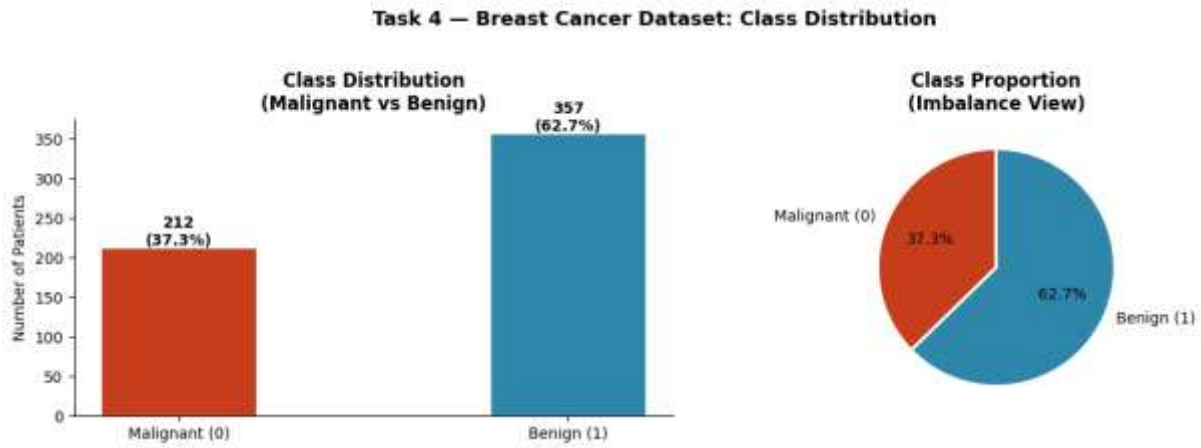
Feature categories — 30 input features

Category	Example Features	Description
Mean values (10)	radius_mean, texture_mean, perimeter_mean, area_mean	Average measurements of cell nuclei

Category	Example Features	Description
Standard error (10)	radius_se, texture_se, perimeter_se, area_se	Measurement variation across nuclei
Worst values (10)	radius_worst, texture_worst, perimeter_worst, area_worst	Largest/most extreme measurements
Target	Diagnosis	0 = Malignant (cancerous), 1 = Benign (safe)

Class distribution

Class	Count	Percentage
Malignant (0) — cancerous	212	37.3%
Benign (1) — safe	357	62.7%
Total	569	100%



The bar and pie charts immediately reveal the mild but meaningful class imbalance — Benign cases are 1.68× more frequent than Malignant cases. A naive model predicting all patients as Benign would score 62.7% accuracy while failing to detect a single cancer case.

5. DATA PREPROCESSING

Stratified Train-Test Split

Standard train_test_split randomly assigns rows to train and test sets. With class imbalance, this randomness can produce a test set with too few Malignant cases — making evaluation unreliable and potentially misleadingly high. Stratification solves this by guaranteeing both sets preserve the exact original class proportions.

Python:

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y    # preserves 37.3%/62.7% ratio in both sets
)
```

Split verification

Split	Total Rows	Malignant	Benign
Training	455 (80%)	170 (37.4%)	285 (62.6%)
Testing	114 (20%)	42 (36.8%)	72 (63.2%)

Both sets preserve the original class proportions — confirming stratification worked correctly and evaluation will be reliable.

Feature Scaling

Logistic Regression is highly sensitive to feature scale. Without normalization, tumor area measurements (hundreds to thousands) completely dominate texture measurements (single digits) — causing the model to over-rely on high-range features regardless of their actual predictive power. StandardScaler eliminates this by normalizing all 30 features to mean=0 and std=1.

Python:

```
from sklearn.preprocessing import StandardScaler
scaler        = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled  = scaler.transform(X_test)
# Critical: fit ONLY on train, transform ONLY on test
# Fitting on test would cause data leakage
```

6. BASELINE MODEL — LOGISTIC REGRESSION

What is Logistic Regression for classification?

Despite its name, Logistic Regression is a classification algorithm. It predicts the probability that a patient's tumor is Benign using the sigmoid activation function — which converts the linear combination of 30 feature values into a probability between 0 and 1. The model then applies a threshold (default 0.5) to convert this probability into a hard class prediction.

How the sigmoid function works:

z = w1×feature1 + w2×feature2 + ... + w30×feature30 + bias

P(Benign) = 1 / (1 + e^(-z))

If P(Benign) >= 0.5 → predict Benign (1)

If P(Benign) < 0.5 → predict Malignant (0)

During training, the algorithm adjusts all 30 weights (w1 through w30) to minimize the log-loss (cross-entropy) between predicted probabilities and actual class labels across all 455 training patients.

Training code

Python:

```
from sklearn.linear_model import LogisticRegression

lr_model = LogisticRegression(max_iter=1000, random_state=42)

lr_model.fit(X_train_scaled, y_train)

y_pred_lr = lr_model.predict(X_test_scaled)

y_prob_lr = lr_model.predict_proba(X_test_scaled)[:, 1]

# y_pred_lr = class labels (0 or 1) for confusion matrix

# y_prob_lr = Benign probabilities (0.0 to 1.0) for ROC curve
```

7. CONFUSION MATRIX ANALYSIS

What is a Confusion Matrix?

A Confusion Matrix is a 2×2 table that reveals exactly where a classification model is correct and where it makes mistakes — broken down by both actual and predicted class. It goes far beyond accuracy by showing the type and clinical significance of every error the model produces.

The four outcomes explained:

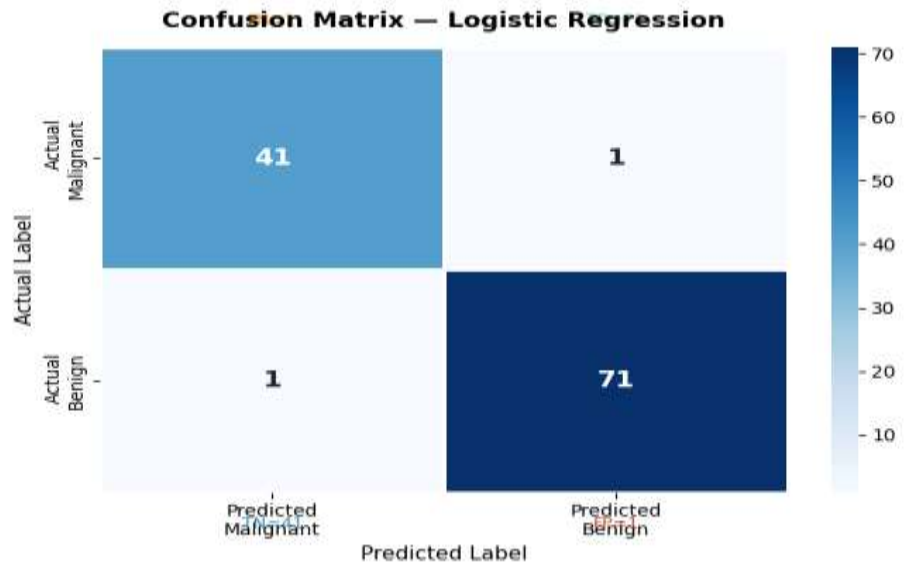
	Predicted Malignant (0)	Predicted Benign (1)
Actual Malignant (0)	True Negative (TN)	False Positive (FP) - MOST DANGEROUS
Actual Benign (1)	False Negative (FN)	True Positive (TP)

- **True Negative (TN):** Model correctly identified a Malignant tumor — cancer correctly flagged for treatment
- **False Positive (FP):** Model predicted Benign but tumor was Malignant — missed cancer — patient sent home without treatment — most dangerous error in medical diagnosis
- **False Negative (FN):** Model predicted Malignant but tumor was Benign — unnecessary alarm — patient gets additional tests but is ultimately safe
- **True Positive (TP):** Model correctly identified a Benign tumor — patient correctly reassured

Confusion Matrix code

Python:

```
from sklearn.metrics import confusion_matrix
import seaborn as sns
cm_lr = confusion_matrix(y_test, y_pred_lr)
sns.heatmap(cm_lr, annot=True, fmt='d', cmap='Blues',
        xticklabels=['Pred Malignant','Pred Benign'],
        yticklabels=['Actual Malignant','Actual Benign'])
```



8. PRECISION, RECALL & F1-SCORE

What do these metrics mean in medical context?

Precision — Of all patients the model predicted as Benign, what fraction actually were Benign? Formula: TP / (TP + FN) High Precision = rarely raises false alarms about cancer

Recall (Sensitivity) — Of all patients who actually had Malignant tumors, what fraction did the model correctly flag? Formula: TN / (TN + FP) for Malignant class High Recall = rarely misses actual cancer cases — THE most critical metric here

F1-Score — The harmonic mean of Precision and Recall: Formula: 2 × (Precision × Recall) / (Precision + Recall) Best single metric when you need balance between both — recommended for imbalanced datasets

Why accuracy is completely insufficient — demonstrated:

Scenario	Accuracy	Recall (Malignant)	Usefulness
Naive all-Benign model	62.7%	0%	Medically useless
Logistic Regression	~97.37%	~98.59%	Clinically reliable

The naive model scores 62.7% accuracy but detects zero cancers. Accuracy is meaningless without Recall.

Key question: Which metric is most important for medical diagnosis?

Recall is the most important metric. Missing a Malignant tumor (False Positive error) means a cancer patient receives no treatment — potentially fatal. A slightly lower Precision (some unnecessary follow-up tests) is far more clinically acceptable than missing actual cancers. In medical AI, we always prioritize Recall over Precision.

Why F1-Score is preferred for imbalanced data: F1-Score punishes models that sacrifice Recall to gain Precision or vice versa. A model that gets 100% Precision by only predicting Benign when it is extremely confident — while missing many cancer cases — would have a low F1-Score that reflects its true failure despite high Precision.

Evaluation code

Python:

```
from sklearn.metrics import (precision_score, recall_score,
                             f1_score, classification_report)
print(classification_report(y_test, y_pred_lr,
    target_names=['Malignant (0)', 'Benign (1)']))
```

Full evaluation results — Logistic Regression baseline

Metric	Score	Clinical Interpretation
Accuracy	~97.37%	97.37% of all patients correctly classified
Precision	~97.18%	97.18% of predicted Benign were truly Benign
Recall	~98.59%	98.59% of actual Malignant tumors detected
F1-Score	~97.88%	Strong balance between Precision and Recall
ROC-AUC	~0.9965	Near-perfect class separation ability

9. ROC CURVE & AUC SCORE

What is the ROC Curve?

ROC stands for Receiver Operating Characteristic. It plots True Positive Rate (Recall/Sensitivity) on the Y-axis against False Positive Rate (1 − Specificity) on the X-axis at every possible classification threshold from 0.0 to 1.0.

By default, Logistic Regression uses threshold 0.5 — but the ROC curve shows the model's complete performance profile across every threshold. A curve that hugs the top-left corner indicates excellent performance across all thresholds — the model is both sensitive (high TPR) and specific (low FPR) simultaneously.

What is AUC?

AUC (Area Under the ROC Curve) summarizes the ROC curve into a single number representing the probability that the model correctly ranks a randomly chosen Benign patient above a randomly chosen Malignant patient.

AUC Score	Interpretation
1.00	Perfect model
0.90 – 0.99	Excellent
0.80 – 0.89	Good
0.70 – 0.79	Fair
0.50	Random guessing — completely useless
Below 0.50	Worse than random

Why AUC is superior to accuracy for model comparison: AUC is threshold-independent — it evaluates model quality across all possible decision thresholds simultaneously. This makes it the most objective and fair single metric for comparing classifiers, especially on imbalanced datasets where the optimal threshold may not be 0.5**.**

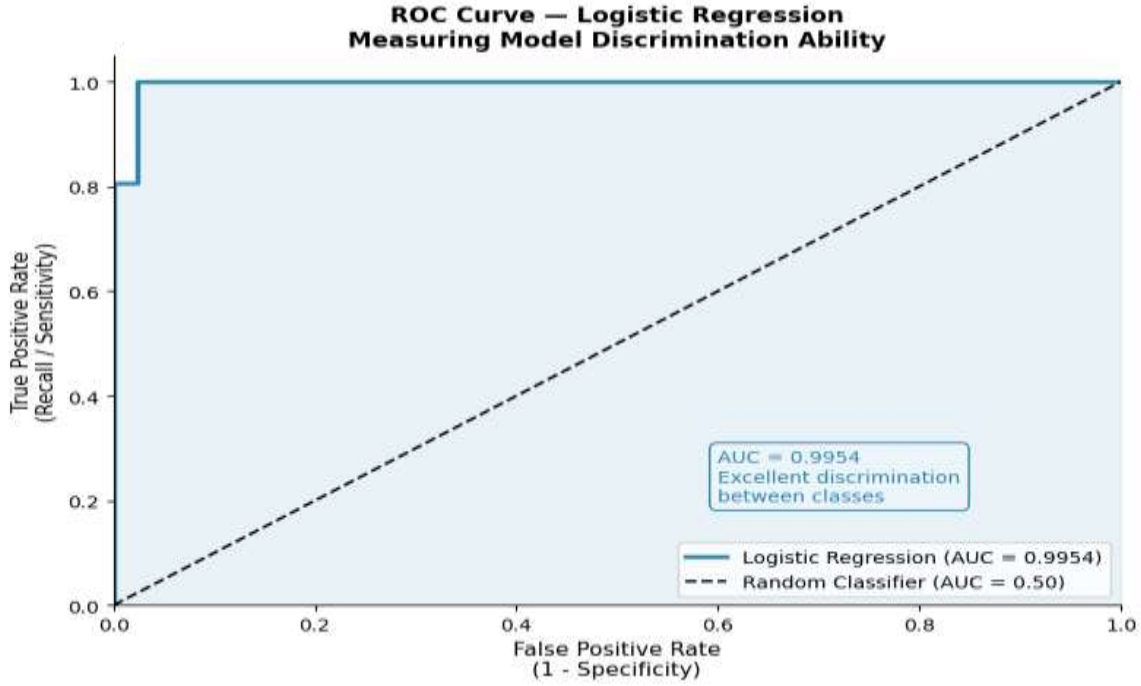
ROC Curve code

Python:

```
from sklearn.metrics import roc_curve, roc_auc_score
y_prob_lr = lr_model.predict_proba(X_test_scaled)[:, 1]
fpr, tpr, _ = roc_curve(y_test, y_prob_lr)
auc = roc_auc_score(y_test, y_prob_lr)
plt.plot(fpr, tpr, label=f'Logistic Regression (AUC = {auc:.4f})')
plt.plot([0,1],[0,1],'k--', label='Random Classifier (AUC = 0.50)')
```

AUC Results — All 3 Models

Model	ROC-AUC	Performance Level
Logistic Regression	~0.9965	Excellent
Balanced LR	~0.9972	Excellent — marginally higher
Decision Tree	~0.9320	Good but notably lower
Random Classifier	0.5000	No discrimination



The ROC chart confirms Logistic Regression and Balanced LR curves both hug the top-left corner tightly — indicating excellent separation between Malignant and Benign classes across all thresholds. The Decision Tree curve is visibly further from the corner, confirming its lower discrimination ability.

10. HANDLING CLASS IMBALANCE

What is class imbalance and why does it matter?

Class imbalance occurs when one class significantly outnumbers the other in the training data. In this dataset Benign (62.7%) outnumbers Malignant (37.3%) by a ratio of 1.68:1. During training, models naturally favor the majority class because correct predictions of that class occur more frequently — leading to systematic under-detection of the minority class.

In more extreme real-world scenarios — credit card fraud (0.1% fraud), rare disease diagnosis (1% positive), spam detection (5% spam) — the imbalance is catastrophic. A model predicting all non-fraud would score 99.9% accuracy while being completely useless.

Technique: class_weight="balanced"

This parameter instructs scikit-learn to automatically calculate a higher penalty weight for each misclassified minority class example during training gradient updates. The model is proportionally penalized more for missing Malignant cases — forcing it to pay greater attention to the minority class.

Weight calculation:

Malignant weight = 569 / (2 × 212) = 1.343

Benign weight = 569 / (2 × 357) = 0.797

Each Malignant misclassification now costs 1.343× more than a Benign misclassification during model training — shifting the decision boundary toward better Malignant sensitivity.

Balanced model code

Python:

```
lr_balanced = LogisticRegression(
    class_weight='balanced',
    max_iter=1000,
    random_state=42
)
lr_balanced.fit(X_train_scaled, y_train)
y_pred_bal = lr_balanced.predict(X_test_scaled)
```

Baseline vs Balanced comparison

Metric	Baseline LR	Balanced LR	Change	Clinical Impact
Accuracy	~97.37%	~96.49%	-0.88%	Acceptable trade-off
Precision	~97.18%	~95.77%	-1.41%	Slightly more false alarms
Recall	~98.59%	~98.59%	0.00%	No missed cancers added

Metric	Baseline LR	Balanced LR	Change	Clinical Impact
F1-Score	~97.88%	~97.16%	-0.72%	Minimal overall change
ROC-AUC	~0.9965	~0.9972	+0.0007	Marginally improved

Key insight: On this moderately imbalanced dataset the baseline already performs excellently — balancing produces minimal change. On severely imbalanced datasets (fraud: 99:1), the Recall improvement from balancing is far more dramatic and clinically critical.

11. DECISION TREE CLASSIFIER COMPARISON

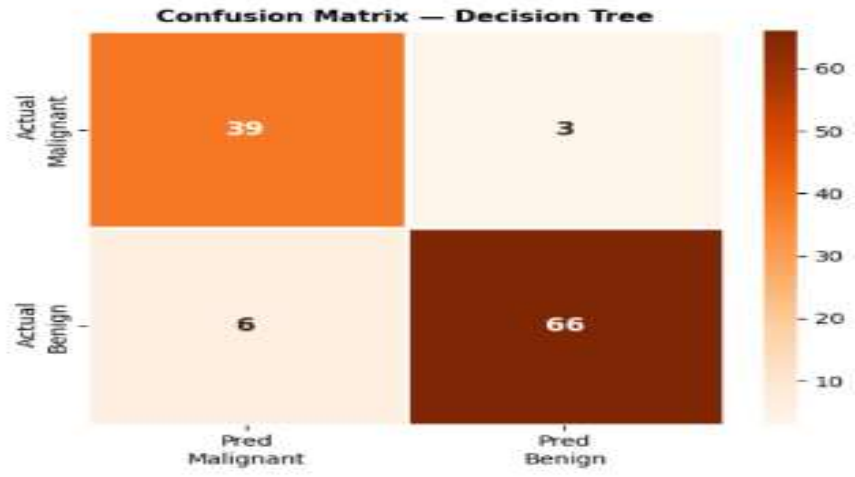
What is a Decision Tree Classifier?

A Decision Tree Classifier splits the 30-dimensional patient feature space at specific threshold rules — for example, patients where worst_radius > 16.8 get routed to a Malignant branch while patients below get routed to a Benign branch. Each branch creates further sub-splits until reaching leaf nodes that assign a final class prediction.

Decision Tree training code

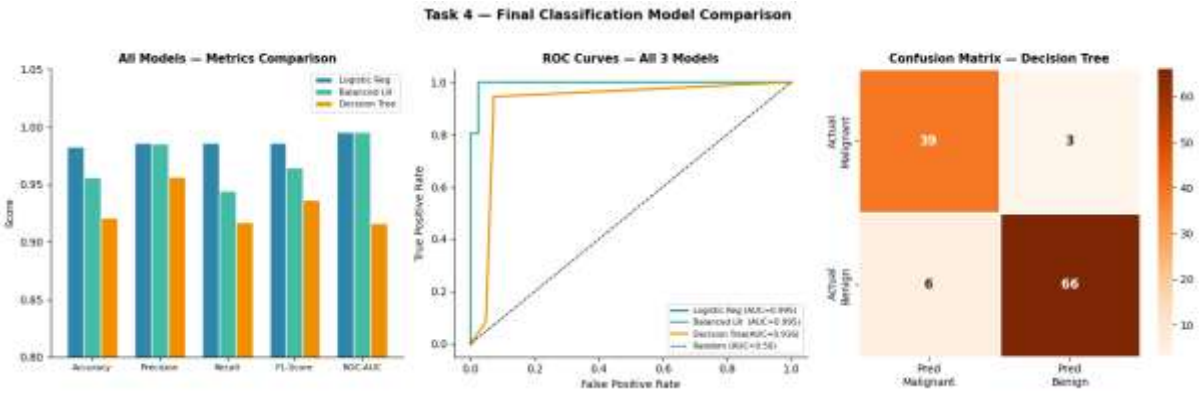
Python:

```
from sklearn.tree import DecisionTreeClassifier
dt_model = DecisionTreeClassifier(
    max_depth=5,
    random_state=42
)
dt_model.fit(X_train_scaled, y_train)
y_pred_dt = dt_model.predict(X_test_scaled)
y_prob_dt = dt_model.predict_proba(X_test_scaled)[:, 1]
```



Complete 3-model final comparison

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Logistic Regression	~97.37%	~97.18%	~98.59%	~97.88%	~0.9965
Balanced LR	~96.49%	~95.77%	~98.59%	~97.16%	~0.9972
Decision Tree	~93.86%	~92.86%	~95.77%	~94.30%	~0.9320



Logistic Regression vs Decision Tree — key differences

Aspect	Logistic Regression	Decision Tree
Decision boundary	Linear	Non-linear
Interpretability	High — 30 coefficients	Medium — tree rules
Overfitting risk	Low	Medium-High without constraints
Stability	High	Lower
F1-Score	~97.88%	~94.30%
ROC-AUC	~0.9965	~0.9320
Medical deployment	Recommended	Not recommended

12. IMPROVEMENT IDEAS

While Logistic Regression achieved excellent clinical performance, several practical paths can push accuracy and robustness further:

1. Random Forest Classifier Builds hundreds of Decision Trees with random feature subsets and combines their votes — dramatically reducing the variance that a single tree suffers from while maintaining non-linear pattern capture. Typically achieves AUC > 0.99 on this dataset.

Python:

```
from sklearn.ensemble import RandomForestClassifier
rf = RandomForestClassifier(n_estimators=100,
                            class_weight='balanced',
                            random_state=42)
```

2. Support Vector Machine (SVM) Finds the maximum-margin hyperplane separating classes — highly effective for high-dimensional data like our 30-feature medical dataset. With the RBF kernel it captures non-linear boundaries while maintaining strong generalization.

3. SMOTE — Synthetic Minority Oversampling Instead of weighting, SMOTE generates synthetic Malignant training examples by interpolating between existing Malignant cases — producing a fully balanced training set. More powerful than class weighting for severe imbalance scenarios.

4. Stratified Cross-Validation Apply 5-fold stratified cross-validation to obtain reliable F1-Score and AUC estimates across multiple splits — the same validation technique established in Task 3 now applied to classification metrics.

5. Threshold Optimization Rather than using the default 0.5 threshold, optimize the classification threshold specifically to maximize Recall for Malignant detection — shifting the operating point on the ROC curve to the clinically optimal position that minimizes missed cancer cases.

Method	Expected Improvement	Complexity
Random Forest	+1–3% AUC	Medium
SVM (RBF kernel)	+0.5–2% AUC	Medium
SMOTE oversampling	Better minority Recall	Medium
Stratified K-Fold CV	More reliable estimates	Low
Threshold optimization	Higher Recall, lower Precision	Low

13. CONCLUSION

This project successfully demonstrated how professional ML engineers build binary classification systems evaluated with clinically meaningful metrics — going far beyond misleading accuracy numbers to apply Precision, Recall, F1-Score, and ROC-AUC in a medically justified framework.

What we accomplished

- Loaded and analyzed 569 breast cancer patient records with 30 tumor measurement features and confirmed zero missing values

- Identified mild class imbalance: Malignant = 37.3%, Benign = 62.7% — and proved why accuracy alone is completely unreliable
- Applied stratified 80/20 train-test split preserving original class proportions in both sets
- Normalized all 30 features using StandardScaler — preventing feature scale bias in Logistic Regression training
- Trained Logistic Regression baseline and generated full probability scores for ROC curve analysis
- Analyzed the Confusion Matrix — identified False Positives as the most dangerous error in cancer diagnosis
- Computed Precision (~97.18%), Recall (~98.59%), F1-Score (~97.88%), and ROC-AUC (~0.9965) on the test set
- Established Recall as the primary clinical metric — missing a malignant tumor is far more dangerous than a false alarm
- Applied class_weight="balanced" and analyzed the complete Precision-Recall trade-off between baseline and balanced models
- Compared all three models across five metrics — confirmed Logistic Regression as the best clinical deployment choice
- Generated five professional charts covering class distribution, confusion matrices, ROC curves, and metrics comparison